# RAG Against the Machine — Study & Build Guide

Nov 5, 2026 · @omayma

## 1. The pipeline in one page

Your grade is overwhelmingly about retrieval, not about the answers. Generation is graded loosely — the subject says outright that a coherent, mostly-grounded answer passes, because Qwen3-0.6B is a tiny model with known reasoning limits. Retrieval is graded with a hard pass/fail number. Spend your effort accordingly.

A language model only knows what was in its training data. It has never seen the vLLM repository on your disk. RAG is the trick that lets it answer questions about that repository anyway: instead of putting knowledge *into* the model, you put the right pages *in front of* the model at the moment it answers.

The subject breaks this into four stages. Learn them as four separate problems — they fail for different reasons.

1. **Indexing** — walk the corpus, cut every file into chunks, turn each chunk into a bag of tokens, build a structure that can score every chunk against a query in milliseconds. Persisted to `data/processed/`. Budget: 5 minutes.
2. **Retrieval** — given a question, score every chunk and return the top-k. Each result is a *source location*: a `file_path` plus a character range. This is the stage with a hard number attached.
3. **Augmenting** — take those top-k chunks, read the real text back off disk from those character ranges, assemble a prompt that fits the model's context window.
4. **Generating** — hand the prompt to `Qwen/Qwen3-0.6B`, get an answer, serialise it as JSON.

### How the stages map onto your six commands

Every command runs as `uv run python -m src <command> [options]`.

| Command | Stage | What it does |
|---|---|---|
| `index --max_chunk_size <int>` | Indexing | Ingest `data/raw/`, write the index to `data/processed/` |

| Command | Stage | What it does |
|---|---|---|
| `search <query> --k <int>` | Retrieval | Top-k sources for one question |
| `search_dataset --dataset_path --k --save_directory` | Retrieval | Top-k for a whole dataset → a `StudentSearchResults` JSON file |
| `answer <query> --k <int>` | Retrieval + generation | Answer one question |
| `answer_dataset --student_search_results_path --save_directory` | Generation | Answers for a dataset → a `StudentSearchResultsAndAnswer` JSON file |
| `evaluate --student_search_results_path --dataset_path` | Your own testing | Your own recall@k, for iterating |

Notice the shape of `answer_dataset`: it does **not** re-run retrieval. It reads the search-results file that `search_dataset` already wrote and only generates answers from it. Retrieval and generation are decoupled on purpose — build them as two independent services that talk through JSON files.

## 2. The grading contract

Read this before you design anything. Every design decision in this project is downstream of how the moulinette scores you.

### What recall@k means here

For each question, the ground truth lists the source locations that genuinely answer it. Recall@k is the share of those correct sources that appear somewhere in your top-k results.

In your public datasets, **every question has exactly one ground-truth source** (I checked all 199). That simplifies the metric enormously: recall@k becomes *the fraction of questions where at least one of your top-k chunks hits the gold span*. It is a hit-or-miss game, one shot per question.

### What counts as a hit

Two conditions, both required:

1. **Same file.** Your `file_path` must equal the reference `file_path` as a string, verbatim. `data/raw/vllm-0.10.1/docs/features/lora.md` matches; `./data/raw/...`, an absolute path, or a path with a different prefix never matches. A result in the wrong file never counts, no matter how good the text is.
2. **Overlapping character range**, with an Intersection-over-Union above **0.05**.

IoU is the overlapped length divided by the combined length:

```
\mathrm{IoU} = \frac{|\text{your span} \cap \text{gold span}|}{|\text{your span} \cup \text{gold span}|}
```

At 0.05 the bar is deliberately low — you do not need to find the exact span, you need to land a chunk that covers roughly the right region of the right file.

## The one trap hidden in that formula

If the gold span sits entirely inside your chunk, the union *is* your chunk, so IoU simplifies to `gold_length / chunk_length`. To clear 0.05 you need `chunk_length < 20 × gold_length`. A huge chunk that swallows a tiny gold span scores zero even though it contains the answer.

I measured this against your public data. At a 2000-character chunk, 96% of docs spans and 98% of code spans still clear the bar; at 1000 characters, 99% of both do. So it costs you a couple of points at most — worth knowing, not worth panicking over. The real argument for smaller chunks is precision, covered in section 6.

## The thresholds you must clear

| Dataset | Metric | Threshold |
|---|---|---|
| Docs questions | Recall@5 | ⩾ 80% |
| Code questions | Recall@5 | ⩾ 50% |

Plus two performance ceilings: **indexing in under 5 minutes** for the whole corpus, and **under 90 seconds for 200 questions** at retrieval time.

### Two hard constraints that silently void your output

- **No retrieved source may exceed 2000 characters.** The moulinette's `max_context_length` is 2000, and a single over-long source makes your *whole output file* invalid — not just that one question. This is why `--max_chunk_size` defaults to 2000 and why you must never exceed it.
- **Never import or call the moulinette** from your code. Your `evaluate` command must be your own implementation of recall@k, written for your own iteration.

### How grading actually runs

At defense the pipeline runs end to end as one automated flow: `index` → `search_dataset` → `./moulinette evaluate_student_search_results`. If your layout or CLI signature does not match the subject exactly, the reference scripts cannot run, and those checks are failed by design. Treat the command names, flag names and directory names as an API contract, not as suggestions.

## 3. BM25 from zero

BM25 is a scoring function. You give it a query and a document; it gives you back a number saying how relevant that document is to that query. You run it over all 13 000 chunks, sort by the number, and take the top-k. That is the whole idea.

The name means "Best Matching 25" — the 25th formula in a research line from the 1990s (Okapi BM25, from the Okapi information-retrieval system). It is *lexical*: it matches literal words, not meaning. It has no neural network, no embeddings, no training. It is arithmetic over word counts, and it is still the default baseline that modern search systems have to beat.

### The problem it solves

Start with the naive idea: score a document by how many query words it contains. Three things immediately go wrong.

**Problem 1: common words dominate.** A question like "How do I configure the OpenAI server?" contains *the*, *do*, *I*. Nearly every chunk in the corpus contains *the*. Matching on it tells you nothing. Rare words like *OpenAI* are the informative ones.

> **Fix — IDF (inverse document frequency).** Weight each query term by how rare it is across the corpus. A term in 3 of 13 000 chunks gets a large weight; a term in 12 000 chunks gets almost none.

**Problem 2: repetition shouldn't scale linearly.** A chunk mentioning *LoRA* 50 times is more about LoRA than one mentioning it twice — but it is not 25× more about it. The first occurrence is strong evidence; the fiftieth adds nearly nothing.

> **Fix — term-frequency saturation.** Let the contribution of a term grow fast at first, then flatten toward a ceiling. This is what the `k1` parameter controls.

**Problem 3: long documents win unfairly.** A 2000-character chunk contains more words than a 300-character chunk, so it matches more query terms by sheer size, not by relevance.

> **Fix — length normalisation.** Penalise chunks that are longer than the corpus average. This is what the `b` parameter controls.

BM25 is exactly these three fixes, combined into one formula. If you understand the three problems, you understand BM25 — the algebra is just bookkeeping.

## The formula

For a query Q made of terms $q_1 \dots q_n$ and a document D:

```
\mathrm{score}(D, Q) = \sum_{i=1}^{n} \mathrm{IDF}(q_i) \cdot \frac{f(q_i, D)
\cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}
{\mathrm{avgdl}}\right)}
```

Read it as: *for each query term, multiply how rare the term is by how strongly this document features it.* Sum over terms.

| Symbol | Meaning |
|---|---|
| `f(q,D)` | How many times term `q` appears in document `D` (the term frequency) |
| `\|D\|` | Length of `D` in tokens |
| `avgdl` | Average document length across the whole corpus |

| Symbol | Meaning |
|---|---|
| `k1` | Saturation knob, typically 1.2–2.0 |
| `b` | Length-normalisation knob, 0 to 1, typically 0.75 |

And IDF, in the standard BM25 (probabilistic) form:

```
\mathrm{IDF}(q) = \ln\!\left(\frac{N - n(q) + 0.5}{n(q) + 0.5} + 1\right)
```

where `N` is the total number of documents and `n(q)` is how many contain `q`. The `+1` inside the log keeps IDF non-negative for terms that appear in more than half the corpus — without it, very common terms get a negative weight and actively push documents down, which is rarely what you want.

## Reading the fraction piece by piece

Look at the term-frequency fraction alone and ignore `b` for a moment (set `b = 0`):

```
\frac{f \cdot (k_1 + 1)}{f + k_1}
```

- At `f = 0` the whole thing is 0 — a term that is absent contributes nothing.
- As `f` grows, the fraction climbs toward a ceiling of `k1 + 1` and never passes it. That is the saturation: the 50th occurrence of *LoRA* barely moves the score.
- **Small `k1`** (say 0.5) saturates almost immediately — BM25 becomes nearly binary, asking only "does this term appear at all?". **Large `k1`** (say 3.0) keeps rewarding repetition for longer.

Now put `b` back. The `k1` in the denominator is multiplied by `(1 - b + b · |D|/avgdl)`:

- **`b = 0`**: that factor is 1, length is ignored entirely.
- **`b = 1`**: full normalisation — a chunk twice the average length needs twice the term frequency to score the same.
- **`b = 0.75`** (the default): mostly normalised, with a little slack. It is the default because it worked best across decades of TREC benchmarks, not because of theory.

## What tuning actually does for you

For this project, `k1 = 1.5, b = 0.75` is a perfectly good starting point. Once your pipeline runs end to end, sweep them on the public datasets — try `k1` in {0.9, 1.2, 1.5, 2.0} and `b` in {0.4, 0.6, 0.75, 1.0} — and keep what scores best. Expect a few points of recall,

not a transformation. Tokenization and chunking (sections 5 and 6) move the number far more.

One useful intuition for this corpus: your chunks are capped at a fixed character size, so they are already fairly uniform in length. That makes `b` less consequential here than it would be over documents ranging from tweets to books. Lowering `b` toward 0.5 sometimes helps when chunks are near-uniform.

### How it is computed in practice

You never literally loop over 13 000 chunks per query. You build an **inverted index**: a dictionary mapping each term to the list of chunks containing it, with their frequencies.

```
"lora"   -> [(chunk_42, 7), (chunk_311, 2), (chunk_980, 1)]
"adapter" -> [(chunk_42, 3), (chunk_77, 5)]
```

To score a query, you look up only the terms in that query, walk only the chunks in those short lists, and accumulate scores in a dictionary. A query of 8 terms touches a few hundred chunks, not 13 000. That is how you hit the 90-seconds-for-200-questions budget with room to spare.

### Library or hand-rolled?

`rank_bm25` is a small pure-Python package with a `BM25Okapi` class; it is simple and widely used, but it scores by iterating over *every* document for every query, which is slower than a real inverted index. `bm25s` is a newer, much faster implementation using sparse matrices.

My honest recommendation: **implement it yourself.** It is roughly 60 lines, you will be asked to explain it at defense, and the subject explicitly warns about handing in code you cannot justify. You also need the inverted index anyway to meet the speed budget. Use `rank_bm25` as a reference to check your scores against, not as your submission.

## 4. TF-IDF vs BM25 — pick BM25

The subject lets you choose either. Choose BM25: it is strictly the better-behaved of the two for this task, and it is no harder to implement.

Both are built on the same two ingredients — term frequency and inverse document frequency. The difference is what they do with term frequency.

| | TF-IDF (classic) | BM25 |
|---|---|---|
| Term frequency | Used raw, or log-damped: `1 + log(f)` | Saturating, with a tunable ceiling via `k1` |
| Document length | Handled indirectly, by L2-normalising the vector | Explicit, tunable via `b` against corpus average |
| Tunable parameters | None, in the usual formulation | `k1` and `b` |
| Mental model | Each document is a vector; score = cosine similarity with the query vector | A probabilistic relevance score, summed per query term |
| Typical retrieval quality | Good | Consistently a little better |

In practice BM25 is TF-IDF with two decades of empirical corrections bolted on. The saturation curve in particular matters for a code corpus: a Python file that uses a variable name 80 times should not bury a documentation page that explains that variable properly.

### If you want to use scikit-learn

`TfidfVectorizer` gives you a working TF-IDF retriever in about ten lines, and it is tempting. Two cautions. First, you still have to override its tokenizer — the default pattern throws away everything that makes code searchable (section 5). Second, you will be asked to explain it, and "scikit-learn did it" is the failure mode the subject's AI-instructions chapter warns about explicitly.

There is a middle path worth knowing: use scikit-learn's `CountVectorizer` with your own tokenizer to build the term-document count matrix fast, then apply the BM25 weighting yourself with NumPy. You get library speed on the boring part and own the part that matters.

## 5. Tokenization — where your score is actually won

This is the most underrated part of the project. BM25 matches tokens. If your question produces tokens that your index never produced, the score is zero no matter how good your formula is. Tokenization is the contract between the two sides.

## The basic pipeline

Text in, list of strings out:

1. **Lowercase.** Questions say "LoRA", code says `lora_request`. Without lowercasing they never meet.
2. **Split into terms.** The naive `re.findall(r"\w+", text)` is where most people stop, and where most people lose points.
3. **Optionally drop stopwords.** *the*, *is*, *how*, *what*. IDF already crushes them to near-zero weight, so the gain is mostly speed, not accuracy. Low priority.
4. **Optionally stem.** Reduce *configuring*, *configured*, *configuration* to a common root. A Porter/Snowball stemmer (from `nltk` or `PyStemmer`) helps on docs questions, which are written in English prose. It can hurt on code, where identifiers are not English words. Consider stemming your markdown index and not your code index — but only after you have a baseline to compare against.

## The code problem, and why it is the whole game

Look at a real question from your `dataset_code_public.json`:

> "What are the default values for FP8_MIN and FP8_MAX constants in vLLM's triton_flash_attention module?"

The gold source is a 93-character span in `vllm/attention/ops/triton_flash_attention.py`. For BM25 to find it, the question's tokens must match tokens in that file. Which depends entirely on how you split identifiers.

Consider `max_model_len` appearing in code, and a question asking about "the max model length":

| Tokenization | Tokens produced | Matches the question? |
|---|---|---|
| Whole identifier only | `max_model_len` | No |
| Split on underscore only | `max`, `model`, `len` | Partly |
| **Both: keep and split** | `max_model_len`, `max`, `model`, `len` | Yes, both ways |

The third row is the technique. **Emit the full identifier *and* its parts.** Then a question that quotes the identifier verbatim matches the whole token, and a question that paraphrases in English matches the parts. The subject hints at exactly this: questions may paraphrase an idea or quote an identifier verbatim, and what you keep at index time decides which of the two you can still match.

The splits you need:

- **snake_case** → split on `_`: `fp8_min` → `fp8`, `min`
- **camelCase / PascalCase** → split on case boundaries: `AsyncLLMEngine` → `async`, `llm`, `engine`
- **dotted paths** → split on `.`: `torch.nn.functional` → `torch`, `nn`, `functional`
- **kebab-case and CLI flags** → `--max-model-len` → `max`, `model`, `len`

A regex worth knowing for the camelCase case: `re.findall(r"[A-Z]+(?![a-z])|[A-Z][a-z]*|[a-z]+|\d+", ident)` handles the acronym boundary correctly, so `HTTPServer` becomes `HTTP`, `Server` rather than `H`, `TTPServer`.

### Why this matters more than k1 and b

Your docs threshold is 80% and your code threshold is 50%. The gap exists because code is genuinely harder: questions are written in English, code is written in identifiers. Every point you gain on the code dataset will come from tokenization and chunking, not from parameter tuning. Build your tokenizer first, test it on a handful of real questions by hand, and only then worry about the formula.

### Keep it symmetric

Use the *same* tokenizer function for indexing and for queries. If you stem at index time you must stem at query time. A mismatch here produces a system that runs perfectly and retrieves nothing — and it is invisible unless you test it. Make this one function, imported in both places, never duplicated.

## 6. Chunking — two mandatory strategies

A chunk is the unit you index and the unit you return. The subject requires **two distinct strategies**, because a Python file and a Markdown page do not break apart the same way.

## Why chunk at all

Three reasons, each independently sufficient:

- A whole file can be 50 000 characters. The moulinette rejects any source over 2000, and one over-long source invalidates your entire output file.
- BM25 over whole files gives you file-level precision when you need region-level precision. A 5000-line file that mentions your query term somewhere is not an answer.
- The retrieved text goes into Qwen3-0.6B's context window, which is small. You can afford a few thousand characters, not a few hundred thousand.

## The non-negotiable invariant

**Every chunk must carry the exact character offsets it occupies in the original file.** `first_character_index` and `last_character_index` are what the grader compares. If you strip whitespace, normalise line endings, or decode with a different encoding than you count with, your offsets drift and every hit becomes a miss — while your system looks like it is working.

Protect yourself: read each file once as text, keep the raw string, and slice chunks out of it by index so that `raw[first:last]` is literally the chunk text. Write a test that asserts exactly that for a random sample of chunks. This single test catches the most expensive class of bug in this project.

Be explicit about encoding too — `open(path, encoding="utf-8", errors="replace")` — and be consistent, because the grader's indices were computed some specific way and a mismatch on a non-ASCII file shifts everything after it.

## Strategy A: Markdown and text

Markdown has a structure you should respect: headings mark topic boundaries, and a chunk that straddles two sections is about two things at once, which dilutes its BM25 score for both.

A reasonable approach:

1. Split the file on heading lines ( `^#{1,6}\s` ).
2. Each section becomes a candidate chunk.
3. If a section exceeds `max_chunk_size` , split it further on paragraph breaks ( `\n\n` ), then on sentences if still too long.

4. If a section is very small, consider merging it with the next one — a 40-character chunk holding just a heading carries almost no signal.
5. Prepend the heading path to the chunk's *indexed text* (not to the stored offsets) so that a chunk under "## LoRA adapters" is searchable by those words even if the body never repeats them.

That last point is a genuine trick: the indexed token bag does not have to be exactly the characters in the span. You can enrich it with the heading trail, the file path words, anything that helps matching — as long as the offsets you *report* still point at the real region.

### Strategy B: Python code

For Python, use the `ast` module. Parse the file, walk the tree, and make a chunk per top-level function, per class, and per method. `ast` gives you line numbers (`node.lineno`, `node.end_lineno`); convert those to character offsets by precomputing the cumulative length of each line.

Why this beats splitting every N characters: a function is a semantic unit. Its name, its docstring, its parameter names and its body all describe the same thing, so they reinforce each other in the term-frequency counts. A fixed-size window cutting a function in half produces two chunks that are each half an idea.

Practical notes:

- Include the decorator lines and the docstring in the chunk — docstrings are prose and match English questions well.
- A function longer than `max_chunk_size` must still be split; fall back to a sliding window over its lines.
- Module-level code outside any function (imports, constants) should still be chunked — your dataset has questions about constants like `FP8_MIN`.
- Wrap `ast.parse` in `try/except SyntaxError` and fall back to the text strategy. Some files in a big repo will not parse (Python 2 remnants, templates, generated code). A crash here kills your whole index.

### Chunk size and overlap

Start at `max_chunk_size = 2000` because that is the default and the ceiling. Then test smaller — 1000 and 500 — and compare recall@5 on both public datasets. Smaller chunks are more precise (the matched terms are concentrated) and clear the IoU bar more easily,

but you need more of them in your top-k to cover the same text.

**Overlap** is worth trying: let consecutive chunks share 10–20% of their characters, so an answer sitting exactly on a boundary is not cut in half. The cost is more chunks, hence a bigger index and slower indexing. Measure it rather than assuming.

The subject explicitly asks you to report the effect of chunk size on your recall@k in the README, so keep a table of your experiments as you go. That table is also the best possible defense material.

## 7. Indexing — what to build and persist

The `index` command walks `data/raw/`, chunks everything, and writes a structure to `data/processed/` that `search` can load and query fast. Budget: **5 minutes** for the whole corpus.

### Which files to index

The subject says to read "the files you judge useful", and that judgment is graded indirectly. Your corpus has 3226 files; only three extensions appear in the public ground truth: `.py` (99 sources), `.md` (97) and `.txt` (3).

Indexing `.json` configs, `.cu` CUDA kernels, `.png` images and build artifacts costs you indexing time and adds chunks that can only ever be false positives. A sensible allow-list: `.py`, `.md`, `.txt`, `.rst`. Skip anything under `.git/`, `__pycache__/`, `build/`, and skip files over a few hundred KB or that fail a UTF-8 decode.

Be prepared to justify this choice at defense — "I measured which file types the questions actually reference" is a strong answer; "I indexed everything" is a weak one, and so is silently dropping file types you never checked.

### What the index contains

Four things:

| Structure | Purpose |
|---|---|
| Chunk metadata | For each chunk id: `file_path`, `first_character_index`, `last_character_index`, token count |
| Inverted index | `term → [(chunk_id, term_frequency), …]` |

| Structure | Purpose |
|---|---|
| Document frequencies | `term → number of chunks containing it`, for IDF |
| Corpus statistics | Total chunk count `N`, average chunk length `avgdl` |

You do not need to store the chunk *text*. Store the offsets and read the text back off disk when you need it for the prompt. That keeps the index small and guarantees the offsets are correct — if the text you show differs from `raw[first:last]`, you have a bug you want to discover.

## Persistence format

JSON is simple and inspectable but slow to parse at this size. Options, roughly in order of how much I would recommend them for this project:

- **Pickle** — trivial, fast, fine for a local project. Load time well under a second.
- **JSON** — human-readable, which is genuinely useful while debugging. Slower, maybe a few seconds to load.
- **SQLite** — overkill here, but elegant if you attempt the incremental-indexing bonus.
- **NumPy `.npz` / sparse matrix** — fastest scoring, more work to build.

Whichever you pick, write a small `Index` class with `save(path)` and `load(path)` so the format is one decision in one place. Also persist the parameters the index was built with (`max_chunk_size`, tokenizer version), so a stale index built with different settings is detectable rather than mysteriously bad.

## Hitting the 5-minute budget

For ~2000 text and Python files this is comfortable in pure Python if you avoid the obvious traps: do not re-read files, do not build strings by concatenation in a loop, do not run a regex you could precompile. Use `collections.Counter` for term frequencies and `defaultdict(list)` for postings.

If it is still slow, profile before optimising — `cProfile` on a 200-file subset will tell you immediately whether the time goes to I/O, `ast` parsing or tokenizing. Multiprocessing over files with `concurrent.futures.ProcessPoolExecutor` is the easy win if you need it, since chunking one file is independent of every other.

Wrap the loop in `tqdm`; the subject requires progress bars for long operations, and the reference screenshot shows two bars (chunking, then tokenizing).

## 8. Retrieval

Given a question, return the top-k source locations. Budget: **200 questions in under 90 seconds**, which is 450 ms per question — very generous for BM25 if you built an inverted index, and impossible if you rebuild the index per query.

### The algorithm

1. Load the index once, at startup. Never per query.
2. Tokenize the question with the *same* tokenizer used at index time.
3. For each query term, look up its postings list and accumulate BM25 contributions into a `defaultdict(float)` keyed by chunk id.
4. Take the k highest scores ( `heapq.nlargest(k, scores.items(), key=itemgetter(1))` — cheaper than sorting everything).
5. Map chunk ids back to `MinimalSource` objects.

A query term absent from the index contributes nothing; just skip it. A query where *every* term is absent yields an empty result list — that is a valid output, not a crash.

### Degenerate inputs that will be tested

The subject says the CLI is tested with edge cases and must never crash with an unhandled traceback. At minimum, handle:

| Input | Correct behaviour |
|---|---|
| Empty query `""` | Return an empty result list, exit cleanly |
| Query of pure punctuation or gibberish | Empty or low-score results, no crash |
| `k = 0` | Empty list |
| `k` larger than the chunk count | Return everything you have, do not pad |
| Negative `k` | Treat as an error with a clear message, not a traceback |

| Input | Correct behaviour |
|---|---|
| Missing index file | Message telling the user to run `index` first |
| Missing or malformed dataset JSON | Clear message, non-zero exit, no traceback |

## Batch mode

`search_dataset` reads a JSON file of questions, runs retrieval for each, and writes one `StudentSearchResults` file. Two details that matter:

- **Preserve `question_id`.** The grader joins your results to the ground truth on it. Carry it through untouched.
- **Scope the output directory by dataset.** The docs and code datasets share the filename `dataset_docs_public.json` / `dataset_code_public.json` across the `UnansweredQuestions` and `AnsweredQuestions` folders; writing every run into one directory overwrites previous results. The subject shows `--save_directory data/output/search_results/UnansweredQuestions`.

## Ideas that help, in rough order of payoff

- **Better tokenization** (section 5). By far the biggest lever, especially on code.
- **Field weighting.** Score the chunk's heading trail, function name and file path separately and add a weighted bonus. A question naming `lora.md` should favour that file.
- **Query expansion.** Split the question's own camelCase/snake_case identifiers the same way you split the corpus. If the question says `max_model_len`, search for the parts too.
- **Separate indexes for code and docs.** You know from the dataset filename which kind of question you are answering. Nothing stops you from maintaining two indexes with different chunking and different parameters, and the 80%/50% split in the thresholds hints that the two populations behave differently. Discuss the trade-off in your README — it is a strong design-decisions section.
- **Deduplication.** If two of your top-5 are overlapping chunks of the same region, you have spent two slots on one answer. Consider collapsing near-duplicate spans before truncating to k.

# 9. Answer generation with Qwen3-0.6B

This stage is graded qualitatively and leniently. The subject is unusually explicit: the criteria describe a target, not a pass/fail bar, and grading prioritises retrieval quality, grounding and prompt strategy over perfect final phrasing. A coherent, mostly-grounded, partially incomplete answer passes.

So: make it work, make it grounded, do not sink days into it.

## What a passing answer is

- **Coherent** — readable natural language.
- **Grounded** — drawn from the retrieved snippets, not invented.
- **On point** — it answers the question that was asked.

## Running the model

`Qwen/Qwen3-0.6B` is a 0.6-billion-parameter model, small enough to run on CPU. The straightforward route is Hugging Face `transformers`: `AutoTokenizer` and `AutoModelForCausalLM`, `.from_pretrained("Qwen/Qwen3-0.6B")`, then `apply_chat_template` to format the messages and `.generate()` to produce text.

Things to get right:

- **Load the model once**, not per question. Model loading dominates runtime otherwise.
- **Qwen3 has a thinking mode.** It can emit `<think>…</think>` blocks before its answer. Either disable it (`enable_thinking=False` in `apply_chat_template`) or strip those blocks from the output before serialising. Leaving raw thinking traces in your `answer` field looks careless.
- **Set a token cap.** `max_new_tokens` around 256–512 keeps generation bounded; a small model left unbounded will ramble.
- **Greedy or low-temperature decoding.** You want faithful extraction, not creativity.
- **`tqdm` over the dataset loop**, and expect this to be slow on CPU. That is fine — there is no time budget on generation, only on indexing and retrieval.

## Building the context

For each question, take the top-k sources from the search-results file, read `raw[first:last]` back off disk for each, and assemble them into the prompt with their file paths as labels. Then truncate to fit.

A 0.6B model's useful context is far smaller than its nominal maximum. Do not paste all 10 sources at full length; take the top 3–5 and budget perhaps 3000–4000 characters of context total. Count tokens with the model's own tokenizer rather than guessing from characters.

### Prompt shape

A system message establishing the rule, then a user message carrying context and question:

```
System: You answer questions about the vLLM codebase using only the
provided sources. If the sources do not contain the answer, say so.
Be concise and factual.

User:
[Source 1: docs/features/lora.md]
<text>

[Source 2: vllm/entrypoints/openai/api_server.py]
<text>

Question: <the question>
Answer:
```

The instruction to admit ignorance matters: it is the single cheapest defense against hallucination, and "grounded, with no major hallucination" is one of the three stated criteria. Mention this choice in your README's design-decisions section.

### Output

Wrap the result in `MinimalAnswer` (which is `MinimalSearchResults` plus an `answer` string), collect them into `StudentSearchResultsAndAnswer`, and write the JSON. The sources you echo back must be the same ones you were given — do not re-run retrieval inside `answer_dataset`.

## 10. Data models and the exact output JSON

All inter-stage data structures must be `pydantic` models. Service classes (indexer, retriever, pipeline) do not have to be.

## The models, and how they nest

```
class MinimalSource(BaseModel):
    file_path: str
    first_character_index: int
    last_character_index: int

class UnansweredQuestion(BaseModel):
    question_id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    question: str

class AnsweredQuestion(UnansweredQuestion):
    sources: List[MinimalSource]
    answer: str

class RagDataset(BaseModel):
    rag_questions: List[AnsweredQuestion | UnansweredQuestion]

class MinimalSearchResults(BaseModel):
    question_id: str
    question: str
    retrieved_sources: List[MinimalSource]

class MinimalAnswer(MinimalSearchResults):
    answer: str

class StudentSearchResults(BaseModel):
    search_results: List[MinimalSearchResults]
    k: int

class StudentSearchResultsAndAnswer(BaseModel):
    search_results: List[MinimalAnswer]
    k: int
```

The inheritance is the thing to notice. `MinimalAnswer` *is* a `MinimalSearchResults` with an answer bolted on; `StudentSearchResultsAndAnswer` *is* `StudentSearchResults` with the richer element type. Your two output files therefore have identical shape except for one extra key per question. If you model that properly, `answer_dataset` becomes: parse the search-results file, add an `answer` to each entry, write it back out.

The subject says you may extend these models. Adding a `score: float` to your internal search results is useful for debugging and ranking. Just make sure the serialised output still validates against the minimal shape — extra fields are permitted, missing ones are not.

## Reading the input datasets

The `UnansweredQuestions` files give you `question_id` and `question`. The `AnsweredQuestions` files additionally give `sources` and `answer` — that is the ground truth, used by the moulinette and by your own `evaluate` command. You run `search_dataset` on the *Unanswered* files and score against the *Answered* ones.

A subtlety in `RagDataset`: `List[AnsweredQuestion | UnansweredQuestion]` is a union, and pydantic will try to coerce to the first matching member. Use `Field(discriminator=...)` or simply parse into the concrete type you expect for each file, rather than relying on union resolution order.

## The output files, exactly

`search_dataset` writes:

```
{
  "search_results": [
    {
      "question_id": "q1",
      "question": "How to configure OpenAI server?",
      "retrieved_sources": [
        {
          "file_path": "data/raw/vllm-
0.10.1/docs/serving/openai_compatible_server.md",
          "first_character_index": 9867,
          "last_character_index": 10100
        }
      ]
    }
  ],
  "k": 10
}
```

`answer_dataset` writes the same with `"answer": "…"` added inside each entry.

Use `model_dump_json(indent=2)` to serialise — it guarantees the output matches the model. Never hand-build these dicts.

### Four ways this output silently fails

1. `file_path` with the wrong prefix — must be exactly `data/raw/vllm-0.10.1/…`, relative to your project root, as ingested.
2. A source span longer than 2000 characters — invalidates the whole file.
3. A missing or altered `question_id` — the join to ground truth fails.
4. `k` not matching the number you were asked for.

Write a validator function that checks all four before you save. Run it every time. It costs ten lines and saves an evaluation.

## 11. CLI, layout and tooling

These are the mechanical requirements. They are easy marks and easy to lose.

### Project layout

The reference exam scripts expect this exactly:

```
.
├── src/                # your module, runnable as python -m src
├── pyproject.toml      # at the root, so uv sync works from the root
├── uv.lock
├── Makefile
├── README.md
└── data/
    ├── raw/                        # the vLLM repo
    ├── processed/                  # your index
    ├── datasets/
    │   ├── UnansweredQuestions/
    │   └── AnsweredQuestions/
    └── output/
        ├── search_results/<DatasetScope>/
        └── search_results_and_answer/<DatasetScope>/
```

`src/` must be runnable as `python -m src <command>`, which means a `src/__main__.py` that hands off to Fire.

## Python Fire

Fire turns a class or a dict of functions into a CLI by reflection. The minimal shape:

```
# src/__main__.py
import fire
from src.cli import Cli

if __name__ == "__main__":
    fire.Fire(Cli)
```

Each public method of `Cli` becomes a subcommand and each parameter becomes a flag. Type hints on the parameters give you Fire's argument parsing for free — which is convenient, since the subject requires type hints everywhere anyway.

Two cautions. Fire is happy to expose anything public on the class, so keep helpers private with a leading underscore. And Fire will surface a raw traceback on an uncaught exception; wrap each command body in `try/except` that prints a clear message and exits non-zero, because the CLI is explicitly tested with bad input.

## Mandatory tooling

| Requirement | Detail |
|---|---|
| Python | 3.10 or later |
| Package manager | `uv` only — the reviewer runs `uv sync` and nothing else |
| Data models | `pydantic` |
| CLI | `python-fire` |
| Progress bars | `tqdm` |
| Model | `Qwen/Qwen3-0.6B` by default |
| Style | `flake8` clean, including bonus files |
| Types | `mypy` clean with the mandatory flags |
| Docstrings | PEP 257, Google or NumPy style, on every function and class |

### The Makefile

Required rules: `install`, `run`, `debug`, `clean`, `lint`, and optionally `lint-strict`. The `lint` rule must run exactly:

```
flake8 .
mypy . --warn-return-any --warn-unused-ignores --ignore-missing-imports \
       --disallow-untyped-defs --check-untyped-defs
```

The Makefile shipped in your archive is a stub — `install` runs `uv install python3`, `clean` runs `rm -rf` with no argument, `debug` is a comment. Rewrite it. `install` should be `uv sync`, `run` should invoke your main script, `debug` should run it under `pdb`, and `clean` should remove `__pycache__` and `.mypy_cache`.

### What not to commit

No large data files, no model weights, no generated outputs. The evaluator generates those. Put `data/raw/`, `data/processed/`, `data/output/` and the model cache in `.gitignore` — but keep the directory structure, with `.gitkeep` files if needed, so the reference scripts find their paths.

## 12. Your actual data — measured

I inspected the corpus and the public datasets in your archive. These numbers should drive your design rather than guesswork.

### The corpus

`vllm-0.10.1`: **3226 files, ~31 MB**. By extension, the ones that matter:

| Extension | Files |
|---|---|
| `.py` | 1763 |
| `.json` | 403 |
| `.md` | 178 |
| `.cu` / `.cuh` | 137 |
| `.sh` | 66 |

| Extension | Files |
|---|---|
| `.yaml` / `.yml` | 79 |
| `.png` | 52 |
| `.txt` | 28 |

## The public datasets

| | docs | code |
|---|---|---|
| Questions | 100 | 99 |
| Ground-truth sources per question | exactly 1 | exactly 1 |
| Source file types | 97 `.md`, 3 `.txt` | 99 `.py` |
| Gold span length, median | 1147 chars | 878 chars |
| Gold span length, range | 40 – 1997 | 12 – 1578 |

## What these numbers tell you

**One source per question.** Recall@k collapses to a hit rate: either one of your top-k lands on the gold span or the question scores zero. There is no partial credit per question. Your effort goes into making the first few results count, not into breadth.

**Only three extensions appear.** `.py`, `.md`, `.txt`. Indexing `.json`, `.cu`, `.png`, `.yaml` adds chunks that can only be false positives, and spends indexing budget. The hidden datasets may differ slightly, so keeping `.rst` and `.txt` as insurance is cheap — but there is no case for indexing images or lockfiles.

**Code sources are exclusively `.py`.** Your Python chunking strategy carries the entire code dataset. The `.cu` and `.cpp` files are never the answer.

**Gold spans are no larger than your chunk ceiling.** The longest is 1997 characters, just under 2000. So the reference spans were almost certainly produced by a chunker much like the one you are about to write, capped at the same 2000. Your chunks and theirs will be of comparable size, which means an IoU of 0.05 is easy to clear whenever you land in the right region.

**The IoU trap is real but small.** Where a gold span sits entirely inside your chunk, IoU is `gold_length / chunk_length`, so you need your chunk under 20× the gold span. Measured against these datasets:

| Your chunk size | docs spans clearing IoU 0.05 | code spans clearing |
|---|---|---|
| 500 | 100% | 99% |
| 1000 | 99% | 99% |
| 1500 | 97% | 99% |
| 2000 | 96% | 98% |

So chunk size costs you at most ~4 points through IoU alone. Choose chunk size for retrieval precision, not out of fear of the overlap rule — but know that 2000 is not free.

### Use both datasets as your test harness

You have 199 questions with known answers. That is a proper evaluation set. Build the loop early: change something, run `search_dataset` on both, run your own `evaluate`, write down the number. Every design question in this guide — chunk size, overlap, stemming, k1, b, field weighting — is settled by that loop, not by argument.

## 13. Study order and build order

### What to learn, in sequence

1. **The RAG loop as a concept** — you have it from section 1. Thirty minutes of reading, no more; the concept is simple and the difficulty is all in the details.
2. **BM25** — section 3. Work through the formula by hand on a toy example: three documents of five words each, one query. Compute IDF, compute the score, see which document wins and why. Do this on paper. It takes twenty minutes and it is the difference between knowing the formula and understanding it.
3. **Inverted indexes** — understand why you look up terms rather than scanning documents. This is also the answer to "how do you make it fast?" at defense.
4. **Tokenization for code** — section 5. Take five real questions from `dataset_code_public.json`, open the gold source file, and check by hand which tokens would match. This exercise will teach you more than any article.

5. **Python's `ast` module** — enough to walk a tree and extract function and class nodes with their line numbers.
6. **Recall@k and IoU** — implement your own `evaluate` early; you cannot improve what you cannot measure.
7. **Pydantic, Fire, tqdm, uv** — tooling, learn as you need it. An hour total.
8. **Hugging Face `transformers` basics** — last, and only enough to load a model and generate text.

## Build order

Build in this sequence so you always have something measurable.

- [ ] Project skeleton: `src/`, `pyproject.toml` via `uv init`, Makefile, `.gitignore`, a Fire CLI with stub commands that print their arguments
- [ ] Pydantic models, all of them, in one `models.py`
- [ ] File walker with the extension allow-list — verify the file count
- [ ] Tokenizer, with identifier splitting — unit-test it on `max_model_len`, `AsyncLLMEngine`, `FP8_MIN`
- [ ] Markdown chunker — test that `raw[first:last]` equals the chunk text, on random samples
- [ ] Python `ast` chunker — same test, plus the `SyntaxError` fallback
- [ ] BM25 index: build, save, load. Time the build; it must be under 5 minutes
- [ ] `search` on a single query — eyeball the results, they should be obviously sane
- [ ] `search_dataset` writing valid `StudentSearchResults` JSON
- [ ] Your own `evaluate` implementing recall@k with the IoU rule — **get your first number here**
- [ ] Run the real moulinette and check your number matches its number. If they disagree, your understanding of the metric is wrong and everything downstream is unreliable
- [ ] Iterate on chunking, tokenization and parameters until docs ≥ 80% and code ≥ 50% at k=5
- [ ] Only now: generation. Load Qwen3-0.6B, build the prompt, `answer` and `answer_dataset`
- [ ] Edge cases: empty query, k=0, missing files, malformed JSON, missing index
- [ ] `make lint` clean under both flake8 and mypy
- [ ] README with all the required sections

- [ ] Full end-to-end rehearsal from a clean clone: `uv sync` → `index` → `search_dataset` → moulinette → `answer_dataset`

### Self-check before defense

You should be able to answer all of these without notes:

- Why does BM25 saturate term frequency, and what does `k1` do to that curve?
- What does `b = 0` mean, and why might it be reasonable with fixed-size chunks?
- Why did you choose BM25 over TF-IDF?
- Why do you emit both the full identifier and its parts?
- Why did you chunk Python with `ast` instead of a fixed window?
- What happens to your recall if you double the chunk size, and why?
- Why do your file paths have to match verbatim?
- What is an inverted index, and why is your search fast?
- Which file types did you index, and how did you decide?
- Where does your system hallucinate, and what did you do about it?

## 14. Resources and README requirements

### What to read

For BM25 and retrieval theory:

- Okapi BM25 — Wikipedia. The formula, the IDF variants, and the `b` extremes (BM11 at `b=1`, BM15 at `b=0`). Start here; it is short and correct.
- The Probabilistic Relevance Framework: BM25 and Beyond — Robertson & Zaragoza, 2009. The authoritative treatment, free PDF. Read sections 1–3 if you want to understand *why* the formula has the shape it does; skim the rest.
- Introduction to Information Retrieval — Manning, Raghavan & Schütze, free online. Chapter 6 for tf-idf and the vector space model, chapter 11 for probabilistic IR and BM25. The standard textbook on this material.

For the tooling:

- `ast` — Python docs. Focus on `ast.parse`, `ast.walk`, and the `lineno` / `end_lineno` attributes.
- Pydantic, Python Fire, uv, tqdm.

- Qwen3-0.6B on Hugging Face — read the model card for the chat template and the thinking-mode flag.
- `rank_bm25` — useful as a reference implementation to validate your own scores against.

### The README the subject requires

It must be in English, at the repository root, and contain:

- **First line, italicised, exactly**: *This project has been created as part of the 42 curriculum by* `<login>`.
- **Description** — the goal and a brief overview.
- **Instructions** — installation and execution.
- **Resources** — references on the topic, *and* a description of how AI was used: which tasks, which parts of the project. This is explicitly required; be specific and honest.
- **System architecture** — your pipeline components and how they interact.
- **Chunking strategy** — your approach to document segmentation, both strategies.
- **Retrieval method** — the algorithm and ranking mechanism.
- **Performance analysis** — your recall@k scores and system performance. This is where the chunk-size experiment table goes.
- **Design decisions** — key implementation choices, with reasons.
- **Challenges faced** — difficulties and how you solved them.
- **Example usage** — clear examples of running your system.

Write the performance-analysis and design-decisions sections *as you build*, not at the end. They are the two sections that distinguish a project that was understood from one that was assembled, and they are the ones your evaluator will use to decide what to ask you.

## 15. Bonus — read later

The bonus is graded **only once the entire mandatory part validates**: every requirement met and the system robust when a reviewer pushes on it. Until then it is not looked at. Do not start any of this before docs ⩾ 80% and code ⩾ 50%, every edge case handled and `make lint` clean.

Five bonuses, one point each, all required to run on a CPU-only campus machine, and each must actually work — describing it in the README does not count, and you may be asked to demonstrate it.

1. **Semantic embeddings** — a vector index built with a lightweight CPU model such as `all-MiniLM-L6-v2`, alongside your lexical index. The natural next step after BM25, and the one that fixes the vocabulary-mismatch problem you will have felt on the docs questions.
2. **Hybrid retrieval** — combine the lexical and semantic rankings into one list. Requires bonus 1. Reciprocal Rank Fusion is the usual way and is about five lines.
3. **Incremental indexing** — when one file changes, re-index only that file. Worth planning for in your index design even if you do not implement it, because it tells you to store per-file metadata.
4. **Caching** — cache the index and query results to speed up cold start and repeated queries.
5. **Local HTTP API** — expose search and answering over a small local HTTP server, so the system can be driven by something other than the CLI. FastAPI makes this an afternoon.

If you do attempt them, bonus 1 and 2 are the pair that belong together and the pair that teach you the most — they are the step from lexical to modern retrieval, and you will meet them again.